

**Universidad Privada Domingo Savio**  
**Facultad de Ingeniería**  
**Ingeniería en Sistemas**



**Configuración de Servidor WEB**

Joaquin Alessandro Felipez Rojas  
Nicolas Sebastian Reguerin Meneses  
David Willy Cruz Huanca

**Asignatura:** Taller de Sistemas Operativos II

**Docente:** Jose Abraham Mita Huanca

**Fecha de entrega:** 17 de septiembre de 2026

**Equipo:** Grupo 1 Sudoers

Cochabamba–Bolivia

2026

# Índice

## CAPÍTULO I

### INTRODUCCIÓN 1

#### 1.1 Introducción 1

## CAPÍTULO II

### OBJETIVOS, ALCANCE Y LÍMITES 2

#### 2.1 Objetivos del trabajo 2

##### 2.1.1 Objetivo general 2

##### 2.1.2 Objetivos específicos 2

#### 2.2 Alcance y Límites 2

##### 2.2.1 Alcance 2

##### 2.2.2 Límites 3

##### 2.2.3 Entorno de trabajo 4

## CAPÍTULO III

### DESARROLLO DEL TRABAJO 5

#### 3.1 Desarrollo del trabajo 5

##### 3.1.1 Arquitectura del servicio 5

##### 3.1.2 Instalación del servicio 6

##### 3.1.3 Generación automática de certificados SSL/TLS 7

##### 3.1.4 Configuración TLS global 8

##### 3.1.5 Soporte WebSocket para Portainer 9

##### 3.1.6 Configuración de los proxies inversos 9

##### 3.1.6.1 Proxy inverso para CUPS (Impresión) 9

##### 3.1.6.2 Proxy inverso para Filebrowser (Archivos) 10

##### 3.1.6.3 Proxy inverso para Portainer (Gestión Docker) 11

##### 3.1.6.4 Proxy inverso para Zabbix (Monitoreo) 12

##### 3.1.6.5 Técnica de resolución dinámica de upstreams 13

##### 3.1.6.6 Cabeceras de proxy compartidas 13

##### 3.1.7 Virtual host del sitio principal 14

##### 3.1.8 Servidor por defecto 15

##### 3.1.9 Definición en Docker Compose 15

|                                                                |           |
|----------------------------------------------------------------|-----------|
| 3.1.10 Pruebas de acceso                                       | 16        |
| 3.1.10.1 Verificación del arranque                             | 16        |
| 3.1.10.2 Prueba del sitio principal                            | 17        |
| 3.1.10.3 Prueba de los proxies inversos                        | 18        |
| 3.1.10.4 Verificación del certificado SSL                      | 19        |
| <b>CONCLUSIONES</b>                                            | <b>21</b> |
| 4. Conclusiones                                                | 21        |
| 4.1 Resultados obtenidos                                       | 21        |
| 4.2 Dificultades encontradas                                   | 21        |
| 4.3 Importancia del servidor web en la administración de redes | 22        |
| 4.4 Aprendizajes adquiridos                                    | 22        |
| <b>REFERENCIAS</b>                                             | <b>24</b> |

## Índice de tablas

|                                                                  |    |
|------------------------------------------------------------------|----|
| <b>TABLA 1:</b> Virtual hosts configurados en Nginx              | 6  |
| <b>TABLA 2:</b> Puertos del servidor web                         | 7  |
| <b>TABLA 3:</b> Características del certificado SSL auto-firmado | 8  |
| <b>TABLA 4:</b> Directivas TLS globales de Nginx                 | 9  |
| <b>TABLA 5:</b> Directivas de resolución dinámica de upstreams   | 13 |
| <b>TABLA 6:</b> Cabeceras HTTP de los proxies inversos           | 13 |
| <b>TABLA 7:</b> Variables de entorno del servidor web            | 16 |

## Índice de figuras

|                                                                                                  |    |
|--------------------------------------------------------------------------------------------------|----|
| <b>FIGURA 1:</b> Arquitectura del servidor web: Nginx como proxy inverso y servidor de contenido | 5  |
| <b>FIGURA 2:</b> Arranque del servidor web Nginx                                                 | 16 |
| <b>FIGURA 3:</b> Sitio web del proyecto accedido desde un navegador                              | 17 |
| <b>FIGURA 4:</b> Acceso a servicios internos a través del proxy inverso                          | 18 |
| <b>FIGURA 5:</b> Verificación del certificado SSL del servidor web                               | 19 |

# **CAPÍTULO I**

## **INTRODUCCIÓN**

## 1.1 Introducción

En toda infraestructura de red institucional, el servidor web constituye la puerta de entrada a los recursos digitales de una organización. A través de los protocolos HTTP y HTTPS, un servidor web atiende las solicitudes de los navegadores de los usuarios, devolviendo páginas, archivos y servicios de forma segura Tanenbaum y Wetherall (2017).

En entornos empresariales, el servidor web frecuentemente cumple funciones adicionales como proxy inverso —redirigiendo las solicitudes de los clientes hacia servicios internos— y terminador TLS —cifrando las comunicaciones con certificados SSL/TLS Nginx, Inc. (2026). Esta capacidad permite centralizar el acceso a múltiples servicios bajo un único punto de entrada, simplificando la administración de la red y mejorando la seguridad al cifrar todas las comunicaciones entre los clientes y los servicios.

El presente informe documenta la configuración de un servidor web dentro del proyecto integrador del curso de Taller de Sistemas Operativos II, desarrollado por el equipo *Grupo 1 Sudoers* en el dominio `sudoers.lan`. Se implementó Nginx, configurado como servidor de contenido estático, proxy inverso hacia los servicios internos del proyecto (impresión, archivos, monitoreo y gestión Docker) y terminador TLS con certificados auto-firmados.

El servicio se despliega sobre un servidor Debian mediante un contenedor Docker, integrándose con el servidor DNS (BIND9) del proyecto para la resolución de nombres dentro del dominio `sudoers.lan`.

Este capítulo introduce el contexto general del taller y presenta la importancia del servidor web en la administración de sistemas informáticos. Los capítulos siguientes desarrollan los objetivos planteados, el alcance y los límites del trabajo, el proceso de configuración del servicio y las conclusiones obtenidas.

## **CAPÍTULO II**

### **OBJETIVOS, ALCANCE Y LÍMITES**

## 2.1 Objetivos del trabajo

### 2.1.1 Objetivo general

Comprender el funcionamiento de un servidor web (Nginx), así como su configuración e integración con los servicios de resolución de nombres (DNS) y los demás servicios de la infraestructura de red del proyecto `sudoers.lan`, en un entorno administrado con Debian.

### 2.1.2 Objetivos específicos

- Configurar un servidor web utilizando Nginx para alojar el sitio institucional del proyecto y servir como proxy inverso hacia los servicios internos.
- Implementar HTTPS con certificados SSL/TLS auto-firmados para la protección de las comunicaciones web.
- Configurar virtual hosts para múltiples subdominios del proyecto (`web.sudoers.lan`, `print.sudoers.lan`, `files.sudoers.lan`, `portainer.sudoers.lan`, `zabbix.sudoers.lan`).
- Configurar proxies inversos hacia los servicios internos del proyecto (CUPS, File-browser, Portainer y Zabbix) con soporte para WebSocket.
- Realizar pruebas de acceso web desde navegadores y verificación de los proxies inversos.
- Documentar el proceso completo de instalación, configuración y prueba del servicio, incluyendo comandos, archivos de configuración y evidencias de cada paso.

## 2.2 Alcance y Límites

### 2.2.1 Alcance

El presente informe documenta la configuración del servidor web del proyecto. Las actividades realizadas incluyen:

- Instalación de Nginx 1.27 sobre Alpine Linux mediante contenedor Docker.
- Configuración de virtual hosts para el sitio principal del proyecto (`web.sudoers.lan`, `www.sudoers.lan`, `sudoers.lan`).
- Configuración de proxy inverso hacia los servicios internos:
  - CUPS (impresión): `print.sudoers.lan`
  - Filebrowser (archivos): `files.sudoers.lan`
  - Portainer (gestión Docker): `portainer.sudoers.lan`
  - Zabbix (monitoreo): `zabbix.sudoers.lan`
- Generación automática de certificados SSL/TLS auto-firmados mediante el script de `entrypoint`.
- Configuración de redirección HTTP a HTTPS en todos los virtual hosts.
- Configuración del servidor por defecto (`default_server`) para capturar peticiones no mapeadas.
- Configuración de soporte WebSocket para Portainer.
- Despliegue de la landing page del proyecto como contenido estático.
- Pruebas de acceso desde navegadores y verificación de los proxies inversos.

### 2.2.2 Límites

Los siguientes aspectos no forman parte de este informe:

- Certificados SSL emitidos por una autoridad de certificación (CA) confiable; se utilizan certificados auto-firmados adecuados para un entorno de laboratorio.
- Integración con dominios externos o configuración de registros DNS públicos.
- Balanceo de carga o alta disponibilidad del servidor web.
- Configuración de WAF (*Web Application Firewall*) o reglas avanzadas de seguridad en Nginx.
- Seguridad avanzada como protección contra ataques DDoS, rate limiting o listas de control de acceso por IP.

### **2.2.3 Entorno de trabajo**

El entorno utilizado para la configuración consta de los siguientes componentes:

- Servidor: máquina física con Debian 12 (Bookworm) donde se ejecutan todos los servicios del proyecto mediante Docker.
- Despliegue: los servicios se ejecutan en contenedores Docker gestionados con Docker Compose, conectados por la red interna `tso-net`.
- Red interna: red administrativa `192.168.0.0/24` (WLO1) y red de usuarios `192.168.20.0/24` (ENO1).
- Servicio: Nginx 1.27 sobre Alpine Linux.
- Resolución de nombres: servidor DNS propio (BIND9) que resuelve el dominio `sudoers.lan` para todos los servicios.
- Clientes de prueba: máquinas Linux y Windows conectadas a la red local, navegadores web para verificación de los sitios y proxies inversos.
- Usuarios: `nicolas`, `joaquin` y `david`, integrantes del equipo *Grupo 1 Sudoers*.

# **CAPÍTULO III**

## **DESARROLLO DEL TRABAJO**

## 3.1 Desarrollo del trabajo

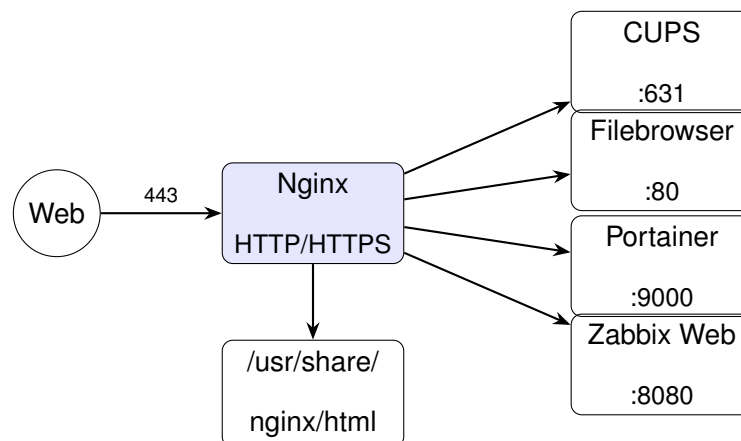
Este capítulo documenta el proceso de configuración del servidor web del proyecto sobre Debian. Se describe desde la instalación del servicio hasta las pruebas de acceso realizadas desde los clientes.

### 3.1.1 Arquitectura del servicio

El servidor web se implementó utilizando Nginx 1.27 sobre Alpine Linux, desplegado como un contenedor Docker. Nginx cumple un doble rol en el proyecto: sirve el sitio web estático del equipo y actúa como proxy inverso hacia los servicios internos de la infraestructura, todo protegido con HTTPS mediante certificados auto-firmados.

El servidor opera como punto de entrada único para el acceso HTTP/HTTPS de todos los servicios del proyecto. La figura 1 presenta la arquitectura general.

**FIGURA 1:** Arquitectura del servidor web: Nginx como proxy inverso y servidor de contenido



**FUENTE:** Elaboración propia.

La tabla 1 resume los virtual hosts configurados y sus destinos.

TABLA 1: Virtual hosts configurados en Nginx

| Subdominio            | Destino             | Función                                |
|-----------------------|---------------------|----------------------------------------|
| print.sudoers.lan     | tso-print:631       | Proxy inverso a CUPS (impresión)       |
| files.sudoers.lan     | tso-files:80        | Proxy inverso a Filebrowser            |
| portainer.sudoers.lan | tso-portainer:9000  | Proxy inverso a Portainer (Web-Socket) |
| zabbix.sudoers.lan    | tso-zabbix-web:8080 | Proxy inverso a Zabbix                 |
| web.sudoers.lan       | Contenido estático  | Sitio principal del proyecto           |
| www.sudoers.lan       | Contenido estático  | Alias del sitio principal              |
| sudoers.lan           | Contenido estático  | Alias del sitio principal              |
| _ (default)           | Contenido estático  | Acceso por IP o dominios no declarados |

FUENTE: Elaboración propia.

### 3.1.2 Instalación del servicio

El servidor web se despliega como un contenedor Docker basado en `nginx:1.27-alpine`. El Dockerfile instala `openssl` como dependencia adicional para la generación de certificados:

```
FROM nginx:1.27-alpine

RUN apk add --no-cache openssl

COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY public/ /usr/share/nginx/html/
COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh

EXPOSE 80 443

ENTRYPOINT ["/entrypoint.sh"]
CMD ["nginx", "-g", "daemon off;"]
```

La tabla 2 resume los puertos expuestos por el contenedor.

TABLA 2: Puertos del servidor web

| Puerto | Protocolo | Servicio                        |
|--------|-----------|---------------------------------|
| 80     | TCP       | HTTP (redirige a HTTPS)         |
| 443    | TCP       | HTTPS (contenido y proxies)     |
| 8080   | TCP       | HTTP alternativo (configurable) |

FUENTE: Elaboración propia.

### 3.1.3 Generación automática de certificados SSL/TLS

El script `entrypoint.sh` genera un certificado SSL auto-firmado al iniciar el contenedor, si no existe uno previo. El certificado cubre el dominio base y todos los subdominios mediante un *wildcard* en el `subjectAltName`:

```
#!/bin/sh
set -e

DOMAIN="${WEB_DOMAIN:-sudoers.lan}"

SSL_DIR="/etc/ssl"
CERT_FILE="${SSL_DIR}/certs/web.pem"
KEY_FILE="${SSL_DIR}/private/web.key"

if [ ! -f "$CERT_FILE" ]; then
    echo ">>> Generando certificado SSL auto-firmado para ${DOMAIN}..."
    mkdir -p "${SSL_DIR}/certs" "${SSL_DIR}/private"
    openssl req -new -x509 -days 3650 -nodes \
        -out "$CERT_FILE" \
        -keyout "$KEY_FILE" \
        -subj "/CN=${DOMAIN}/O=TSO-11/C=AR" \
        -addext "subjectAltName = DNS:${DOMAIN}, DNS:*.${DOMAIN}" \
        2>/dev/null
    chmod 600 "$KEY_FILE"
    echo ">>> Certificado generado: ${CERT_FILE}"
fi
```

```
else
    echo ">>> Certificado SSL ya existe , omitiendo generacion"
fi

exec "$@"
```

El certificado generado tiene las siguientes características:

**TABLA 3:** Características del certificado SSL auto-firmado

| Parámetro                 | Valor                              |
|---------------------------|------------------------------------|
| Tipo                      | X.509 v3, auto-firmado             |
| Validez                   | 3650 días (10 años)                |
| CN ( <i>Common Name</i> ) | sudoers.lan                        |
| Organización              | TSO-II                             |
| País                      | AR                                 |
| subjectAltName            | DNS:sudoers.lan, DNS:*.sudoers.lan |
| Clave privada             | Sin passphrase (-nodes)            |

**FUENTE:** Elaboración propia.

### 3.1.4 Configuración TLS global

La configuración TLS se aplica a nivel global en `nginx.conf`, habilitando los protocolos TLS 1.2 y TLS 1.3 con cache de sesión compartido:

```
ssl_protocols TLSv1.2 TLSv1.3;
ssl_prefer_server_ciphers off;
ssl_session_cache shared:SSL:10m;
ssl_session_timeout 1d;
```

**TABLA 4:** Directivas TLS globales de Nginx

| Directiva                                  | Justificación                                                                                |
|--------------------------------------------|----------------------------------------------------------------------------------------------|
| <code>ssl_protocols</code>                 | Solo se aceptan TLS 1.2 y 1.3, descartando versiones obsoletas e inseguras                   |
| <code>ssl_prefer_server_ciphers off</code> | Se permite que el cliente elija el cifrado preferido, siguiendo las recomendaciones modernas |
| <code>ssl_session_cache</code>             | Cache compartido de 10 MB para reutilizar sesiones TLS entre conexiones                      |
| <code>ssl_session_timeout</code>           | Las sesiones expiran tras 1 día, equilibrando rendimiento y seguridad                        |

**FUENTE:** Elaboración propia.

### 3.1.5 Soporte WebSocket para Portainer

Portainer requiere conexiones WebSocket para la interacción en tiempo real con la interfaz de gestión. Nginx implementa este soporte mediante un bloque `map` que transforma las cabeceras de actualización:

```
map $http_upgrade $connection_upgrade {
    default upgrade;
    ''      close;
}
```

Este bloque asigna el valor `upgrade` a la variable `$connection_upgrade` cuando el cliente envía la cabecera `Upgrade` (típica de WebSocket), y `close` en caso contrario.

### 3.1.6 Configuración de los proxies inversos

Cada servicio interno del proyecto se expone a través de un subdominio con redirección HTTP a HTTPS y resolución dinámica del upstream. A continuación se documenta la configuración de cada proxy.

#### 3.1.6.1 Proxy inverso para CUPS (Impresión)

El servidor de impresión CUPS se expone en `print.sudoers.lan`:

```
server {
    listen 80;
```

```
server_name print.sudoers.lan;

return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name print.sudoers.lan;

    ssl_certificate      /etc/ssl/certs/web.pem;
    ssl_certificate_key  /etc/ssl/private/web.key;

    location / {
        resolver 127.0.0.11 valid=10s ipv6=off;
        set $cups_upstream http://tso-print:631;
        proxy_pass $cups_upstream;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

### 3.1.6.2 Proxy inverso para Filebrowser (Archivos)

El servicio de archivos Filebrowser se expone en `files.sudoers.lan` con un límite de subida de 500 MB:

```
server {
    listen 443 ssl;
    server_name files.sudoers.lan;

    ssl_certificate      /etc/ssl/certs/web.pem;
    ssl_certificate_key  /etc/ssl/private/web.key;
```

```
client_max_body_size 500M;

location / {
    resolver 127.0.0.11 valid=10s ipv6=off;
    set $files_upstream http://tso-files:80;
    proxy_pass $files_upstream;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
}
```

### 3.1.6.3 Proxy inverso para Portainer (Gestión Docker)

Portainer se expone en `portainer.sudoers.lan` con soporte WebSocket y timeouts extendidos de 1 hora:

```
server {
    listen 443 ssl;
    server_name portainer.sudoers.lan;

    ssl_certificate      /etc/ssl/certs/web.pem;
    ssl_certificate_key  /etc/ssl/private/web.key;

    location / {
        resolver 127.0.0.11 valid=10s ipv6=off;
        set $portainer_upstream http://tso-portainer:9000;
        proxy_pass $portainer_upstream;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

```
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection $connection_upgrade;
    proxy_read_timeout 3600;
    proxy_send_timeout 3600;
}
}
```

#### 3.1.6.4 Proxy inverso para Zabbix (Monitoreo)

El panel web de Zabbix se expone en `zabbix.sudoers.lan`:

```
server {
    listen 443 ssl;
    server_name zabbix.sudoers.lan;

    ssl_certificate      /etc/ssl/certs/web.pem;
    ssl_certificate_key  /etc/ssl/private/web.key;

    location / {
        resolver 127.0.0.11 valid=10s ipv6=off;
        set $zabbix_upstream http://tso-zabbix-web:8080;
        proxy_pass $zabbix_upstream;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

### 3.1.6.5 Técnica de resolución dinámica de upstreams

Todos los bloques de proxy inverso comparten una técnica fundamental: la resolución dinámica del upstream mediante una variable y el resolver interno de Docker:

```
resolver 127.0.0.11 valid=10s ipv6=off;
set $upstream http://nombre-contenedor:puerto;
proxy_pass $upstream;
```

**TABLA 5:** Directivas de resolución dinámica de upstreams

| Directiva           | Justificación                                                                              |
|---------------------|--------------------------------------------------------------------------------------------|
| resolver 127.0.0.11 | Utiliza el servidor DNS embebido de Docker para resolver nombres de contenedores           |
| valid=10s           | Refresca la resolución cada 10 segundos, adaptándose a cambios de IP en los contenedores   |
| ipv6=off            | Desactiva la resolución IPv6 para evitar errores en redes sin soporte                      |
| set \$variable      | Al usar una variable, Nginx resuelve el upstream en tiempo de ejecución, no en el arranque |

**FUENTE:** Elaboración propia.

Esta técnica es esencial para la resiliencia del servidor web: si un backend no está disponible al momento de arrancar Nginx, el servidor no falla sino que resuelve el nombre cuando llega la primera solicitud.

### 3.1.6.6 Cabeceras de proxy compartidas

Todas las configuraciones de proxy inverso incluyen las mismas cabeceras HTTP para garantizar que los servicios internos reciban la información correcta del cliente original:

**TABLA 6:** Cabeceras HTTP de los proxies inversos

| Cabecera          | Función                                      |
|-------------------|----------------------------------------------|
| Host              | Envía el nombre de host original del cliente |
| X-Real-IP         | IP real del cliente (no la del proxy)        |
| X-Forwarded-For   | Cadena de proxies intermedios                |
| X-Forwarded-Proto | Protocolo original (HTTP o HTTPS)            |

**FUENTE:** Elaboración propia.

### 3.1.7 Virtual host del sitio principal

El sitio web del proyecto se sirve como contenido estático desde `/usr/share/nginx/html`, respondiendo a tres nombres de dominio:

```
server {  
    listen 80;  
    server_name web.sudoers.lan www.sudoers.lan sudoers.lan;  
    return 301 https://$host$request_uri;  
}  
  
server {  
    listen 443 ssl;  
    server_name web.sudoers.lan www.sudoers.lan sudoers.lan;  
  
    ssl_certificate      /etc/ssl/certs/web.pem;  
    ssl_certificate_key  /etc/ssl/private/web.key;  
  
    root /usr/share/nginx/html;  
    index index.html;  
  
    location / {  
        try_files $uri $uri/ =404;  
    }  
}
```

El contenido estático se monta como volumen de solo lectura desde el host, lo que permite actualizar la página sin reconstruir la imagen:

```
volumes:  
  - ${WEB_SITE_DIR:-./services/web/public}:/usr/share/nginx/html:ro
```

### 3.1.8 Servidor por defecto

El servidor por defecto captura cualquier petición que llegue por IP directa o por un dominio no declarado en la configuración:

```
server {  
    listen 80 default_server;  
    server_name _;  
    return 301 https://$host$request_uri;  
}  
  
server {  
    listen 443 ssl default_server;  
    server_name _;  
  
    ssl_certificate      /etc/ssl/certs/web.pem;  
    ssl_certificate_key  /etc/ssl/private/web.key;  
  
    root /usr/share/nginx/html;  
    index index.html;  
  
    location / {  
        try_files $uri $uri/ =404;  
    }  
}
```

El bloque `default_server` garantiza que ninguna petición HTTP quede sin respuesta, redirigiendo siempre a HTTPS y mostrando el sitio principal.

### 3.1.9 Definición en Docker Compose

El servicio web se declara en el archivo `docker-compose.yml` con la siguiente configuración:

---

```

web:
  build: ./services/web
  container_name: tso-web
  ports:
    - "80:80"
    - "443:443"
    - "${WEB_PORT_HTTP:-8080}:80"
  environment:
    WEB_DOMAIN: ${WEB_DOMAIN:-sudoers.lan}
  volumes:
    - ${WEB_SITE_DIR:-./services/web/public}:/usr/share/nginx/html:ro
  networks:
    - tso-net

```

TABLA 7: Variables de entorno del servidor web

| Variable      | Default               | Descripción                          |
|---------------|-----------------------|--------------------------------------|
| WEB_DOMAIN    | sudoers.lan           | Dominio base para el certificado SSL |
| WEB_PORT_HTTP | 8080                  | Puerto HTTP alternativo en el host   |
| WEB_SITE_DIR  | ./services/web/public | Directorio del contenido estático    |

FUENTE: Elaboración propia.

### 3.1.10 Pruebas de acceso

#### 3.1.10.1 Verificación del arranque

Se comprueba que Nginx arrancó correctamente y que el certificado SSL fue generado:

```

$ docker compose up -d --build web
$ docker compose logs web
>>> Generando certificado SSL auto-firmado para sudoers.lan ...
>>> Certificado generado: /etc/ssl/certs/web.pem

```

FIGURA 2: Arranque del servidor web Nginx

**CAPTURA PENDIENTE**

Guardar como: captura-web-arranque.png

Captura de pantalla mostrando los logs de arranque del contenedor tso-web con la generación del certificado SSL.

**FUENTE:** Captura de pantalla mostrando los logs de arranque del contenedor tso-web con la generación del certificado SSL.

**3.1.10.2 Prueba del sitio principal**

Desde el host o un cliente de la red, se accede al sitio principal:

```
# Acceso por HTTP (debe redirigir a HTTPS)
$ curl -I http://web.sudoers.lan
HTTP/1.1 301 Moved Permanently
Location: https://web.sudoers.lan/

# Acceso por HTTPS (certificado auto-firmado, se usa -k)
$ curl -k https://web.sudoers.lan
<!DOCTYPE html>
...
```

**FIGURA 3:** Sitio web del proyecto accedido desde un navegador

**CAPTURA PENDIENTE**

Guardar como: captura-web-sitio.png

Captura de pantalla del navegador mostrando la landing page del proyecto  
sudoers.lan con certificado HTTPS.

**FUENTE:** Captura de pantalla del navegador mostrando la landing page del proyecto sudoers.lan con certificado HTTPS.

**3.1.10.3 Prueba de los proxies inversos**

Se verifica que cada subdominio redirige correctamente al servicio interno correspondiente:

```
# Proxy a CUPS (impresion)
$ curl -k https://print.sudoers.lan

# Proxy a Filebrowser (archivos)
$ curl -k https://files.sudoers.lan

# Proxy a Portainer (gestion Docker)
$ curl -k https://portainer.sudoers.lan

# Proxy a Zabbix (monitoreo)
$ curl -k https://zabbix.sudoers.lan
```

**FIGURA 4:** Acceso a servicios internos a través del proxy inverso

**CAPTURA PENDIENTE**

Guardar como: captura-web-proxy.png

Captura de pantalla del navegador mostrando el acceso exitoso a uno de los servicios internos a través del proxy inverso de Nginx.

**FUENTE:** Captura de pantalla del navegador mostrando el acceso exitoso a uno de los servicios internos a través del proxy inverso de Nginx.

**3.1.10.4 Verificación del certificado SSL**

Se comprueba que el certificado auto-firmado está correctamente generado con los datos esperados:

```
$ openssl s_client -connect web.sudoers.lan:443 \  
    -servername web.sudoers.lan \  
    </dev/null 2>/dev/null | openssl x509 -noout -subject -issuer -  
    dates  
subject=CN=sudoers.lan , O=TSO-II , C=AR  
issuer=CN=sudoers.lan , O=TSO-II , C=AR  
notBefore = ...  
notAfter = ... (10 años)
```

**FIGURA 5:** Verificación del certificado SSL del servidor web

**CAPTURA PENDIENTE**

Guardar como: `captura-web-cert.png`

Captura de pantalla mostrando la salida de `openssl s_client` con los datos del certificado auto-firmado.

**FUENTE:** Captura de pantalla mostrando la salida de `openssl s_client` con los datos del certificado auto-firmado.

## 4. Conclusiones

La implementación del servidor web permitió comprender de manera integral el funcionamiento de este servicio fundamental en una infraestructura de red. A continuación se presentan las conclusiones más relevantes.

### 4.1 Resultados obtenidos

El servidor web fue implementado exitosamente con las siguientes características verificadas:

- Nginx operativo como servidor de contenido estático y proxy inverso, atendiendo solicitudes HTTP y HTTPS en los puertos 80 y 443.
- Certificado SSL/TLS auto-firmado generado automáticamente al iniciar el contenedor, cubriendo el dominio base y todos los subdominios del proyecto mediante *wildcard*.
- Redirección HTTP a HTTPS funcional en todos los virtual hosts, garantizando que las comunicaciones se cifren.
- Cinco proxies inversos configurados y operativos: CUPS (impresión), Filebrowser (archivos), Portainer (gestión Docker), Zabbix (monitoreo) y el sitio principal.
- Soporte WebSocket para Portainer con timeouts extendidos.
- Resolución dinámica de upstreams mediante el DNS embebido de Docker, garantizando la resiliencia del servidor ante reinicios de backends.
- Servidor por defecto (`default_server`) que captura peticiones por IP o dominios no declarados.

### 4.2 Dificultades encontradas

Durante la configuración se encontraron las siguientes dificultades:

- Certificados auto-firmados: el uso de certificados SSL auto-firmados genera avisos de seguridad en los navegadores web, requiriendo la aceptación manual de excepciones. En un entorno de producción se recomienda utilizar certificados emitidos por

una autoridad de certificación confiable.

- Resolución de upstreams en el arranque: Nginx falla si no puede resolver un nombre de upstream al iniciar. Se solucionó utilizando variables y el resolver de Docker (127.0.0.11), que permite la resolución en tiempo de ejecución.
- Soporte WebSocket para Portainer: la interfaz de Portainer requiere conexiones WebSocket persistentes. Sin la configuración de las cabeceras `Upgrade` y `Connection`, la interfaz no funciona correctamente.
- Tamaño de subida en Filebrowser: el límite predeterminado de Nginx para el cuerpo de las solicitudes (1 MB) era insuficiente para la subida de archivos. Se solucionó con `client_max_body_size 500M`.

### 4.3 Importancia del servidor web en la administración de redes

El servidor web aporta los siguientes beneficios a la administración de sistemas:

- Punto de entrada único: como proxy inverso, centraliza el acceso a todos los servicios del proyecto bajo un único dominio con múltiples subdominios.
- Seguridad: la terminación TLS en Nginx cifra todas las comunicaciones entre los clientes y los servicios internos, incluso cuando los backends no implementan cifrado propio.
- Flexibilidad: los virtual hosts permiten agregar nuevos servicios sin modificar la infraestructura de red ni abrir puertos adicionales en el firewall.
- Presencia institucional: el sitio web estático del proyecto proporciona una interfaz profesional accesible desde cualquier navegador.
- Simplicidad operativa: al concentrar la configuración TLS y el enrutamiento en un solo punto, se simplifica la administración de certificados y la gestión del acceso a los servicios.

### 4.4 Aprendizajes adquiridos

El taller permitió desarrollar competencias prácticas en administración de servicios de red:

- Se comprendió el rol del servidor web como proxy inverso y terminador TLS, consolidando el acceso a múltiples servicios bajo una única interfaz HTTPS.
- Se aprendió a configurar virtual hosts en Nginx para servir múltiples subdominios con distintos backends.
- Se implementó la generación automática de certificados SSL auto-firmados, comprendiendo los conceptos de clave pública, `subjectAltName` y cadenas de confianza.
- Se comprendió la importancia de la resolución dinámica de upstreams en entornos contenerizados, donde las direcciones IP de los servicios pueden cambiar entre reinicios.
- Se aprendió a configurar soporte WebSocket en Nginx para servicios que requieren comunicación bidireccional en tiempo real.
- Se valoró la infraestructura como código: toda la configuración del servidor web quedó definida en archivos versionados, permitiendo su reproducción exacta en cualquier entorno.

## Referencias

- Debian Project. (2026). *Debian system and network administration guide*. <https://www.debian.org/doc/manuals/debian-reference/>. (Recurso de referencia para administración de sistemas Debian y prácticas de configuración.)
- Docker, Inc. (2026). *Docker documentation: Networking*. <https://docs.docker.com/network/>. (Documentación oficial de Docker sobre redes, incluyendo el DNS embebido 127.0.0.11.)
- Nginx, Inc. (2026). *Nginx documentation*. <https://nginx.org/en/docs/>. (Documentación oficial del servidor web Nginx, incluyendo configuración de proxy inverso y TLS.)
- Stewart, J., y Chapple, M. (2017). *Cissp: Certified information systems security professional study guide* (7.<sup>a</sup> ed.). Indianapolis, IN: Wiley.
- Tanenbaum, A. S., y Wetherall, D. J. (2017). *Redes de computadoras* (6.<sup>a</sup> ed.). Madrid: Pearson.